

Assignment # 01

TCS504-System Design

Last date of Submission: 24/08/2026

Note: Students are required to complete this assignment sincerely and submit it on time. For any queries, students may discuss them with the faculty members teaching the System Design course. After the submission of the assignment, an offline quiz based on this assignment will be conducted in the class.

Question 1: Monolithic Breakdown & Latency Bottleneck Analysis.

Scenario: ShopCart is an e-commerce platform (like Amazon) built as a single monolithic application. During a flash sale, traffic spikes **40x** for 2 hours, making the entire app unresponsive. Unrelated pages like "View Order History" also time out.

Peak Traffic Metrics: CPU Utilization: 35% | Memory Utilization: 40% | Database Utilization: 92% | Average Response Time: 3.8s.

The development team suggests adding more application servers (horizontal scaling) to resolve the issue.

Tasks:

1. Explain why a monolithic architecture causes an unrelated feature (Order History) to fail during a flash sale. Propose an N-tier / service-separated architecture to isolate this failure mode.
2. Evaluate the team's suggestion to add more application servers. Will it solve the problem? Identify the exact system bottleneck using the provided latency and throughput metrics.
3. Given checkout latency is measured at 180ms per request (where DB disk seek is 0.1–1ms and same-datacenter network trip is 0.5ms), identify the dominant latency contributor and propose the **first three architectural changes** you would implement to optimize it.

Hints:

1: A monolithic system is an application in which most or all major functionalities are developed, deployed, and run as a single application unit.

2:

Metric	Value
Traffic increase	40×
CPU utilization	35%
Memory utilization	40%
Database utilization	92%
Average response time	3.8 seconds

Check the numbers to get indication of the bottleneck.

3: Can 180 ms checkout latency explainable by a 0.1–1 ms disk seek or 0.5 ms network trip?

Question 2: Protocol Selection & Communication Overhead.

Scenario: You are designing **RidersChoice**, a large-scale ride-hailing platform similar to Uber or Lyft.

RideNow connects:

- **Riders** who request rides
- **Drivers** who accept and complete rides
- **Dispatchers** who monitor and manage operations
- **Travel partners** who access RideNow data
- **Internal microservices** that perform matching, pricing, payments, routing, notifications, etc.

At the same time, RideNow has an **internal low-latency mathematical calculation engine** that performs extremely fast calculations such as:

- Dynamic/surge pricing
- Fare calculation
- Driver-rider matching scores
- ETA calculations
- Route scoring
- Demand prediction
- Distance calculations etc.

Tasks:

1. Assign the most appropriate protocol (REST, RPC/gRPC, GraphQL, or WebSockets) to each of the following 4 interactions. For each choice, explain why the other three protocols would be a worse fit:
 - Interaction A: Rider app receiving continuous live updates of driver locations.
 - Interaction B: Internal "Matching Service" calling "Pricing Service" thousands of times per second inside the same datacenter.
 - Interaction C: External travel partners querying ride history with dynamic, unpredictable fields.
 - Interaction D: Real-time collaborative driver route planning where multiple dispatchers edit the map simultaneously.
2. For internal microservice communication (Matching ↔ Pricing), explain specifically why gRPC/RPC is more efficient than REST over HTTP/JSON. Mention at least two concrete costs (e.g., serialization overhead, connection management) that REST adds.

Hints: Choose the protocol according to the communication pattern, not simply because one protocol is universally "faster."

Question 3: Load Balancing Failures & Consistent Hashing

Scenario: StreamHub places a Layer 4 (L4) Load Balancer with a Round Robin algorithm in front of 4 video-streaming servers. In production, two critical issues occur:

- Issue A: One server gets overloaded handling long-lived streaming connections, while others handling short video metadata requests sit idle.
- Issue B: A personalization feature requires user sessions to stick to the server that cached their watch history, but Round Robin keeps routing the same user to different servers.

Tasks:

1. Diagnose why Round Robin fails in Issue A. Recommend a better load-balancing algorithm, explain its underlying mechanism, and identify one

scenario where your chosen algorithm could fail.

2. To solve Issue B using Consistent Hashing:

- Explain how Consistent Hashing maps users to servers and why it limits cache invalidation to $1/N$ of keys when a server node crashes (compared to $\text{Hash}(\text{key}) \bmod N$).
- Explain how Virtual Nodes prevent hotspots on the hash ring.

3. Would switching from an L4 to an L7 Load Balancer Resolve Issue A or Issue B? Justify your answer based on visibility at Layer 4 vs. Layer 7.

Hints:

1. L4 load balancer operates primarily with transport-level information such as Source IP, Destination IP, Source port, Destination port, TCP/UDP connection state. It generally does not understand the application-level meaning of the traffic. The number of connections may not be a reasonable approximation of workload.
2. Servers are perfectly placed to handle the equal loads.
3. Application layers load balancers have the information about HTTP methods, URL/paths, headers, cookies etc.

Question 4: Caching Strategies, Eviction Policies & Stampedes

Scenario: NewsFeedX is a social media application where feeds are read 500 times for every 1 write (500:1 read-to-write ratio). Feed data can tolerate 30 seconds of staleness, but post "like counts" update rapidly and must remain near real-time. During viral spikes, the fixed-size Redis cache evicts viral posts, causing a massive Cache Stampede (50,000 concurrent requests falling through to the database).

Tasks:

1. Select and justify the best reading strategy (Cache-Aside vs. Read-Through) for the feed data based on the 500:1 read: write ratio.
2. Select a write strategy (Write-Through, Write-Back, or Write-Around)

specifically for post "like counts". Explain the exact risk/trade-off you are accepting with your choice.

3. Compare LRU (Least Recently Used) vs. LFU (Least Frequently Used) eviction policies for a post going viral mid-spike. Explain precisely why LRU might evict a viral post and why LFU is (or isn't) the better choice.

Hints.

1. Architecture should optimize for very high read volume. The application therefore has more control over what gets cached and how it is regenerated. Request coalescing, also known as single-flight, is a performance pattern that merges multiple identical concurrent requests into a single backend execution. The first caller triggers the fetch or calculation, while duplicate incoming requests wait for that exact execution to finish and share its result.